ROADRESQ
Backup & Disaster Recovery Plan

Version 1.0 • Data Protection, Business Continuity & Service Recovery

Scope: PostgreSQL/PostGIS • Redis • Object Storage • Application • Configuration • Infrastructure • Operational Data
Objective: Protect critical data and restore safe RoadResQ operations after failure, corruption, outage or disaster

PROTECT → DETECT → RESTORE → VALIDATE → RECOVER → LEARN

Document Control

Field	Value
Document	Backup & Disaster Recovery Plan
Product	RoadResQ
Version	1.0
Status	Production backup and recovery baseline
Audience	Engineering, DevOps, Security, QA, Support and Operations
Related documents	Deployment & DevOps Plan • Production Operations Runbook • Monitoring & Observability Plan • System Architecture • Security Threat Model

1. Purpose

This plan defines how RoadResQ protects critical data, detects backup failures, recovers from infrastructure and application disasters, validates restored systems, and returns the platform to safe operation. The plan covers both routine recovery and major disaster scenarios such as database corruption, accidental deletion, compromised infrastructure, regional outage and loss of a production environment.

2. Recovery Objectives

Recovery targets must ultimately be approved by the business based on customer impact, financial risk and operational capability. The following are recommended starting targets for production planning:

Tier	Examples	Suggested RTO	Suggested RPO
Tier 0 — Critical	Booking state, core API, payment records	≤ 1 hour	≤ 15 minutes
Tier 1 — Important	Provider profiles, vehicles, service history	≤ 4 hours	≤ 1 hour
Tier 2 — Supporting	Analytics, non-critical operational data	≤ 24 hours	≤ 24 hours
Tier 3 — Rebuildable	Caches, ephemeral sessions, temporary artifacts	Rebuild	Not applicable

Important: These are planning targets, not guarantees. Final RTO/RPO values require capacity, provider and cost validation.

3. Disaster Recovery Principles

• PostgreSQL/PostGIS is the authoritative source for transactional business state.

• Redis is treated as recoverable ephemeral/coordination infrastructure, not the sole source of critical data.

• Backups must be isolated from the primary production environment.

• A backup is not considered reliable until restoration has been tested.

• Recovery procedures must be documented, repeatable and auditable.

• Financial records require reconciliation after recovery.

• Security incidents require evidence preservation before destructive remediation.

• Recovery should prioritize safe customer/provider operations over restoring every secondary feature simultaneously.

4. Data Classification

Data class	Examples	Protection priority
Critical transactional	Bookings, job cards, estimates, invoices, payments	Highest
Operational	Providers, vehicles, availability, notifications, support tickets	High
Sensitive documents	Provider verification documents, uploaded evidence	High
Realtime/ephemeral	Redis state, WebSocket sessions, temporary locks	Recover/rebuild
Analytics/derived	Aggregates, reports, non-authoritative dashboards	Medium
Temporary	Caches, temporary uploads, transient processing data	Low/rebuildable

5. Backup Architecture

Recommended model:

```
PRIMARY PRODUCTION → AUTOMATED BACKUP → SEPARATE BACKUP STORAGE → SECONDARY/IMMUTABLE COPY → PERIODIC RESTORE TEST
```

• Use managed PostgreSQL backups where available.

• Enable point-in-time recovery when supported and justified.

• Maintain an independent backup copy outside the primary failure domain.

• Use encryption at rest and in transit.

• Restrict backup access to authorized operational identities.

• Enable retention/lifecycle policies.

• Consider immutable/WORM-style protection for critical backups against ransomware or malicious deletion.

6. PostgreSQL/PostGIS Backup Strategy

Backup type	Purpose	Recommended baseline
Continuous WAL/PITR	Fine-grained recovery	Enable where supported
Daily full/snapshot	Primary recovery point	Daily
Weekly retained backup	Longer recovery history	Weekly
Monthly archival	Long-term protection	Monthly where required
Pre-change backup	High-risk migrations	Before major schema/data changes

Backups must include the PostgreSQL/PostGIS data required to reconstruct the authoritative application state. Extension/configuration dependencies should be documented as part of recovery testing.

7. Redis Backup & Recovery

• Use managed Redis persistence only where the workload requires it.

• Critical booking/payment state must not depend solely on Redis persistence.

• Document which Redis keys/data structures are disposable and which require recovery.

• After Redis loss, rebuild caches and ephemeral state from PostgreSQL/application state.

• Re-establish queues and distributed coordination safely.

• Monitor Redis recovery for stale locks or duplicated work.

8. Object Storage Backup

• Keep production buckets private.

• Enable versioning where appropriate.

• Use lifecycle policies for temporary files.

• Maintain a recovery/replication strategy for important provider documents and customer-uploaded evidence.

• Protect deletion operations with appropriate permissions.

• Test restoration of representative objects and metadata.

• Do not store database secrets or infrastructure credentials as ordinary application files.

9. Configuration & Infrastructure Backup

• Version-control infrastructure definitions and deployment manifests.

• Maintain encrypted backups of required non-secret configuration metadata.

• Keep secret material in an approved secret manager; recovery must include a documented secret restoration/rotation procedure.

• Preserve DNS, domain, certificate, deployment and environment configuration documentation.

• Keep infrastructure state protected and recoverable where IaC state is used.

10. Backup Schedule

Asset	Frequency	Retention concept	Verification
PostgreSQL PITR/WAL	Continuous	Provider/policy defined	Automated health
PostgreSQL full	Daily	7–30+ days	Automated job check
Weekly DB backup	Weekly	1–3+ months	Periodic restore
Long-term backup	Monthly	Business/legal policy	Periodic restore
Object storage	Continuous/versioned or scheduled	Policy defined	Sample restore
Infrastructure definitions	Every change via Git	Long-term	Repository integrity
Critical configuration metadata	On change + scheduled	Policy defined	Recovery drill

11. Backup Monitoring

• Alert when scheduled backups fail.

• Alert when backup age exceeds the permitted recovery point.

• Monitor backup storage capacity.

• Monitor replication/PITR health where applicable.

• Track last successful backup timestamp.

• Track restore-test success/failure.

• Detect unauthorized backup deletion or access anomalies.

• Include backup health in the production operations dashboard.

12. Backup Security

• Encrypt backups at rest and during transfer.

• Use separate credentials/roles for backup operations.

• Restrict restore privileges more tightly than ordinary application access.

• Use MFA for privileged human access.

• Audit backup creation, deletion and restoration.

• Separate production and backup environments/accounts where practical.

• Protect encryption keys according to the organization's key-management policy.

• Test recovery after credential rotation.

13. Restore Validation

A successful backup job does not prove recoverability. Every production backup strategy must include restoration tests.

Restore test flow:

```
SELECT RECOVERY POINT → RESTORE ISOLATED DATABASE → VERIFY SCHEMA → VERIFY COUNTS/CONSTRAINTS → RUN APPLICATION TESTS → VALIDATE CRITICAL JOURNEYS → RECORD RESULTS
```

• Validate users, vehicles, providers and bookings.

• Validate booking status history and relationships.

• Validate invoices/payments and reconciliation identifiers.

• Validate PostGIS/geospatial functionality.

• Validate representative object-storage files.

• Record actual restore duration and data-loss window.

14. Recovery Environment

• Maintain documented infrastructure definitions sufficient to recreate application services.

• Keep a separate recovery/staging environment available for restore validation.

• Use production-like configuration without exposing restored sensitive data unnecessarily.

• Keep recovery access restricted.

• Maintain tested procedures for DNS, certificates, secrets and external integration configuration.

15. Disaster Scenarios

Scenario	Impact	Primary recovery strategy
Accidental deletion	Data loss	Point-in-time restore + validate affected records
Database corruption	Integrity risk	Restore known-good point + reconcile
Ransomware/compromise	System/data trust lost	Isolate, preserve evidence, recover from clean backup
Cloud/region outage	Service unavailable	Failover/alternate environment according to architecture
Application deployment failure	Service degradation	Rollback/redeploy previous artifact
Redis loss	Realtime/cache/queue disruption	Rebuild ephemeral state safely
Object storage loss	Media/documents unavailable	Versioned/replicated restore
Credential compromise	Unauthorized access	Contain, rotate secrets, assess impact, recover if necessary

16. Major Disaster Response

```
DETECT → DECLARE INCIDENT → PROTECT EVIDENCE → CONTAIN → ASSESS BLAST RADIUS → SELECT RECOVERY POINT → RESTORE → VALIDATE → RECONNECT TRAFFIC → MONITOR → CLOSE
```

• Assign Incident Commander and technical recovery lead.

• Freeze unrelated production changes.

• Identify the last known-good recovery point.

• Preserve security evidence if compromise is suspected.

• Recover critical services first.

• Do not reconnect untrusted/compromised components without validation.

• Document every major recovery decision.

17. Database Point-in-Time Recovery

For supported PostgreSQL environments, point-in-time recovery should be the preferred method for accidental deletion or recent corruption when the required recovery window is available.

PITR procedure:

```
IDENTIFY INCIDENT TIME → SELECT SAFE RECOVERY TIMESTAMP → RESTORE TO ISOLATED INSTANCE → VALIDATE → DETERMINE CORRECTIVE DATA SET → PROMOTE/REPLACE ACCORDING TO APPROVED PROCEDURE
```

Never overwrite the only surviving database copy during investigation.

18. Application Recovery

• Deploy the last known-good immutable application artifact.

• Restore compatible database state.

• Restore required configuration/secrets through controlled mechanisms.

• Verify health/readiness endpoints.

• Verify authentication.

• Verify booking creation/matching/state transitions.

• Verify provider operations and realtime updates.

• Verify payment integration and reconciliation.

• Monitor error rates and business metrics after traffic restoration.

19. Booking Data Integrity After Recovery

• Compare booking counts and status distributions against expected records.

• Check for impossible state transitions.

• Check active bookings that were in progress during the incident.

• Identify duplicate requests or assignments.

• Validate provider/customer relationships.

• Review job cards, estimates and invoices connected to affected bookings.

• Use audited correction workflows rather than direct destructive edits.

20. Payment Recovery & Reconciliation

Payments require a separate reconciliation pass because the payment gateway and RoadResQ database can temporarily disagree during a disaster.

• Freeze automated financial actions if integrity is uncertain.

• Export/obtain authoritative gateway transaction status through approved mechanisms.

• Match gateway transaction/reference IDs with RoadResQ payment records.

• Identify paid, failed, pending and unknown transactions.

• Resolve discrepancies through controlled reconciliation.

• Prevent duplicate capture/refund operations.

• Document financial impact and recovery actions.

21. Realtime & Queue Recovery

• Restart WebSocket/realtime services only after core API/database state is stable.

• Allow clients to reconnect and reconcile authoritative booking state.

• Rebuild Redis queues from durable records where the application supports it.

• Review in-flight jobs for duplicates before replay.

• Use idempotency keys and job identifiers to prevent duplicate business effects.

• Monitor queue age and realtime event delivery after recovery.

22. External Dependency Recovery

Dependency	Recovery consideration
Maps/routing	Validate credentials, quota and fallback behavior
Payment	Verify gateway status and webhook/reconciliation state
OTP/SMS	Confirm provider connectivity and avoid uncontrolled resend storms
Push/email	Verify credentials and delivery after application recovery
Object storage	Validate bucket access, versioning and restored objects

23. Traffic Restoration

Do not immediately restore full traffic after a major disaster. Use controlled restoration where infrastructure supports it.

• Validate recovery environment.

• Route a small/controlled amount of traffic if possible.

• Monitor errors, latency and database load.

• Validate critical customer/provider journeys.

• Increase traffic gradually.

• Keep incident monitoring active until stability is confirmed.

24. Data Loss Handling

• Determine exact recovery point and estimated data-loss interval.

• Identify affected bookings, payments, users or providers.

• Do not silently fabricate missing records.

• Use external system records, audit logs and operational evidence for reconciliation where appropriate.

• Communicate material impact according to incident/privacy/legal policy.

• Record permanent loss and corrective actions.

25. Recovery Verification Checklist

Area	Validation
Infrastructure	Instances/services healthy
Database	Schema, data, constraints and performance healthy
Redis	Available; ephemeral state rebuilt
API	Health/readiness and critical endpoints pass
Web	Accessible; no abnormal client errors
Workers	Queues processing normally
Realtime	Connections/events functioning
Bookings	Create/match/status/completion paths valid
Payments	Gateway connectivity + reconciliation validated
Notifications	Critical transactional delivery functioning
Storage	Representative files accessible
Monitoring	Dashboards/alerts receiving telemetry
Security	Access controls/secrets validated

26. Backup & Recovery Testing Program

Test	Frequency concept	Goal
Automated backup verification	Daily	Detect failed jobs
Sample restore	Monthly	Prove backup usability
Full recovery drill	Quarterly	Validate RTO/RPO and procedure
Regional/major disaster exercise	Semiannual/annual	Validate alternate recovery capability
Security recovery exercise	Periodic	Validate clean recovery after compromise
Post-incident restore review	After relevant incident	Capture real-world gaps

27. Recovery Drill Procedure

```
PLAN → ANNOUNCE TEST → SELECT BACKUP → RESTORE ISOLATED ENVIRONMENT → RUN AUTOMATED TESTS → RUN CRITICAL JOURNEYS → MEASURE RTO/RPO → DOCUMENT GAPS → REMEDIATE
```

• Use sanitized or appropriately controlled data in non-production recovery environments.

• Measure actual recovery time rather than estimating it.

• Test secrets/configuration restoration as part of full drills.

• Verify DNS/traffic and external integrations where included in the drill scope.

28. Ransomware / Compromise Recovery

• Isolate compromised systems and credentials.

• Do not destroy compromised evidence.

• Identify the last known-clean backup.

• Validate backup integrity before restoration.

• Rotate affected credentials and secrets.

• Rebuild from trusted artifacts rather than assuming compromised hosts are clean.

• Perform security validation before returning to production.

• Review access logs and determine notification obligations.

29. Recovery Dependencies

Dependency	Required for recovery
Source repository	Application source + infrastructure definitions
Container registry	Known-good application artifacts
Database backup/PITR	Authoritative transactional state
Object storage backup/versioning	Documents/media
Secret manager	Runtime credentials/configuration
DNS/domain	Traffic routing
TLS certificates	Secure access
External service accounts	Maps/payment/notification integrations
Observability	Recovery verification and incident visibility

30. Business Continuity Priorities

When full restoration is not immediately possible, prioritize:

• Customer safety messaging and support.

• Active emergency assistance bookings.

• Provider dispatch and assignment.

• Booking state integrity.

• Payment correctness and reconciliation.

• Customer/provider authentication and access.

• Core communication/notifications.

• Non-critical analytics and secondary features last.

31. Recovery Communication

• Maintain a single authoritative incident record.

• State what is known, what is affected and what is being done.

• Avoid unsupported recovery-time promises.

• Notify customer/provider support teams when workflows are degraded.

• Use approved customer-facing messaging for material outages.

• Record significant communications in the incident timeline.

32. Roles During Recovery

Role	Responsibility
Incident Commander	Overall coordination and prioritization
Recovery Lead	Technical recovery execution
Database Lead	DB restore/integrity/reconciliation
DevOps	Infrastructure, deployment, traffic and secrets
Backend	Application/business-state validation
QA	Recovery verification and critical journey tests
Security	Compromise assessment, containment and clean-room recovery
Support/Ops	Customer/provider impact handling
Finance/Ops	Payment reconciliation and financial impact

33. Backup Cost & Retention Management

• Use lifecycle policies to balance recovery requirements and storage cost.

• Keep critical backups longer than disposable telemetry.

• Review database/object growth regularly.

• Avoid redundant backups without a defined recovery purpose.

• Document legal/business retention requirements separately from technical recovery requirements.

34. Recovery Security Controls

• Recovery environments must use restricted access.

• Restored production data must not become broadly accessible.

• Use encrypted channels and storage.

• Rotate credentials when compromise is suspected.

• Audit restore operations.

• Validate application authorization after recovery.

• Do not disable security controls merely to accelerate recovery.

35. Recovery Readiness Checklist

Control	Ready?
Automated database backups	■
Point-in-time recovery where required	■
Separate backup storage	■
Object storage recovery/versioning	■
Infrastructure definitions recoverable	■
Secrets recovery/rotation procedure	■
Backup failure alerts	■
Restore tests completed	■
RTO/RPO approved	■
Critical journey recovery test	■

Control	Ready?
Payment reconciliation procedure	■
Security compromise recovery procedure	■
Recovery roles/escalation defined	■
Runbook synchronized with live infrastructure	■

36. Definition of Done — Backup & DR

• Critical data has an automated, isolated backup strategy.

• Backup success/failure is monitored.

• Restore procedures are documented and tested.

• RTO/RPO targets are approved and measurable.

• Application and infrastructure can be recreated from version-controlled artifacts.

• Database, object storage, configuration and secrets have recovery procedures.

• Payment and booking integrity are explicitly validated after recovery.

• Security compromise recovery includes clean rebuild and credential rotation.

• Recovery drills produce tracked corrective actions.

• The team can restore RoadResQ without relying on undocumented individual knowledge.

37. Final Recovery Standard

RoadResQ disaster recovery is successful only when the platform returns to a trustworthy state—not merely when servers are online. Recovery must restore authoritative data, preserve booking and payment integrity, re-establish secure access, reconnect critical integrations, validate customer/provider journeys and provide measurable evidence that operations are safe to resume.